JAVA

Java

- Java is a **programming language** and a **platform**. Java is a high level, robust, object-oriented and secure programming language.
- Java was developed by *Sun Microsystems* (which is now the subsidiary of Oracle) in the year 1995. *James Gosling* is known as the father of Java. Before Java, its name was *Oak*. Since Oak was already a registered company, so James Gosling and his team changed the Oak name to Java.
- **Platform**: Any hardware or software environment in which a program runs, is known as a platform. Since Java has a runtime environment (JRE) and API, it is called a platform.

Applications of java

According to Sun, 11 billion devices run Java. There are many devices where Java is currently used. Some of them are as follows:

- Desktop Applications such as acrobat reader, media player, antivirus, etc.
- Web Applications such as irctc.co.in, javatpoint.com, etc.
- Enterprise Applications such as banking applications.
- Mobile
- Embedded System
- Smart Card
- Robotics
- Games, etc.

Types of Java Applications

There are mainly 4 types of applications that can be created using Java programming:

1) Standalone Application

Standalone applications are also known as desktop applications or window-based applications. These are traditional software that we need to install on every machine. Examples of standalone application are Media player, antivirus, etc. AWT and Swing are used in Java for creating standalone applications.

2) Web Application

An application that runs on the server side and creates a dynamic page is called a web application.
Currently, Servlet, JSP, Struts, Spring, Hibernate, JSF, etc. technologies are used for creating web applications in Java.

3) Enterprise Application

An application that is distributed in nature, such as banking applications, etc. is called enterprise application. It has advantages of the high-level security, load balancing, and clustering. In Java, EJB is used for creating enterprise applications.

4) Mobile Application

An application which is created for mobile devices is called a mobile application. Currently, Android and Java ME are used for creating mobile applications.

JVM(Java Virtual Machine)

The **Java Virtual Machine (JVM)** is a virtual environment that executes Java bytecode. It acts as an intermediary between the Java program and the operating system, making Java platform independent.

Functions of JVM

- Loads Java class files.
- Verifies bytecode for security.
- Converts bytecode into machine code using the Just-In-Time (JIT) compiler.
- Executes the program.
- Manages memory (Heap, Stack).
- Performs Garbage Collection.
- Handles exceptions and thread management.

JRE (Java Runtime Environment)

The **Java Runtime Environment (JRE)** provides the environment required to **run** Java applications. It contains the JVM along with the libraries and supporting files needed for execution.

Components of JRE

- JVM
- Core Java Libraries
- Supporting Files
- Configuration Files

Uses

- Runs Java applications.
- Does not contain development tools like the Java compiler.

Note

- A user who only wants to execute Java programs needs only the **JRE**.

JDK (Java Development Kit)

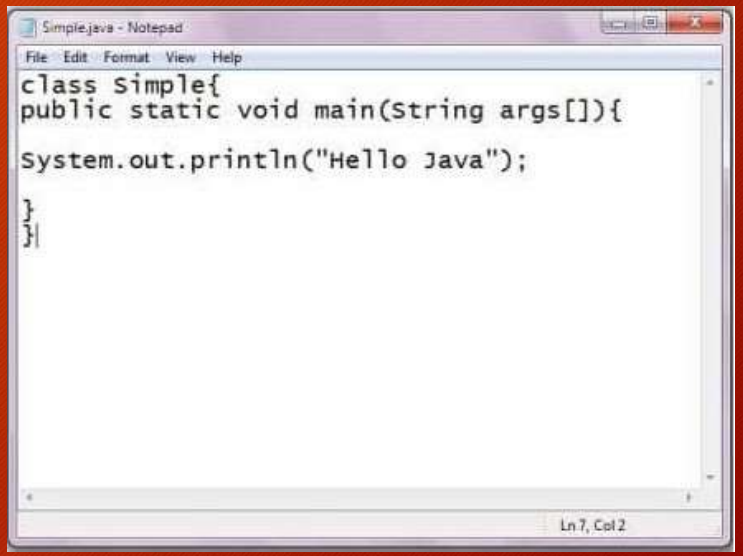
The **Java Development Kit (JDK)** is a complete software development kit used to **develop, compile, debug, and run** Java applications.

Components of JDK

- JRE
- JVM
- Java Compiler (javac)
- Debugger (jdb)
- Documentation Generator (javadoc)
- Archiver (jar)
- Java Launcher (java)

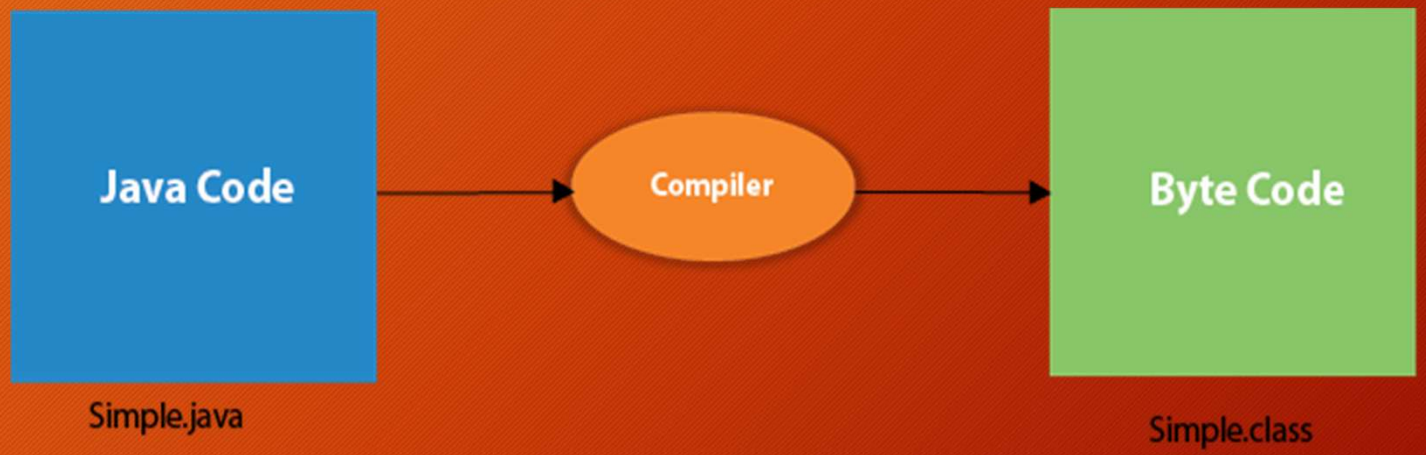
Parameters used in First Java Program

- **class** keyword is used to declare a class in Java.
- **public** keyword is an access modifier that represents visibility. It means it is visible to all.
- **static** is a keyword. If we declare any method as static, it is known as the static method. The core advantage of the static method is that there is no need to create an object to invoke the static method. The main() method is executed by the JVM, so it doesn't require creating an object to invoke the main() method. So, it saves memory.
- **void** is the return type of the method. It means it doesn't return any value.
- **main** represents the starting point of the program.
- **String[] args** or **String args[]** is used for command line argument. We will discuss it in coming section.
- **System.out.println()** is used to print statement. Here, System is a class, out is an object of the PrintStream class, println() is a method of the PrintStream class.
- To write the simple program, you need to open notepad by **start menu -> All Programs -> Accessories -> Notepad** and write a simple program as we have shown here:



Compilation Flow:

- When we compile Java program using javac tool, the Java compiler converts the source code into byte code.



- As displayed in the above diagram, write the simple program of Java in notepad and saved it as Simple.java. In order to compile and run the above program, you need to open the command prompt by **start menu -> All Programs -> Accessories -> command prompt**. When we are done with all the steps properly, it shows the following output:



Bytecode

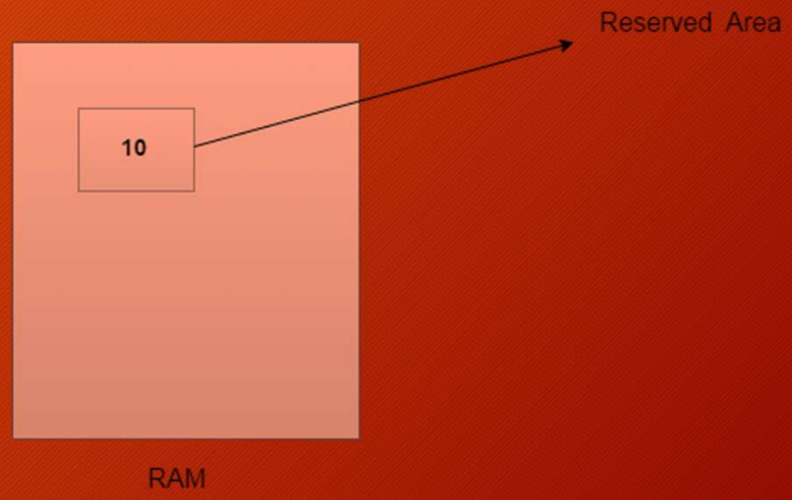
Bytecode is the intermediate code generated by the Java compiler (**javac**) after compiling a Java source file (.java). It is stored in a **.class** file.

Why Bytecode?

- Instead of generating machine code directly, Java generates bytecode.
- Advantages:
- Platform Independent
- Secure
- Compact
- Easy to verify
- Can run on any JVM

Java Variables

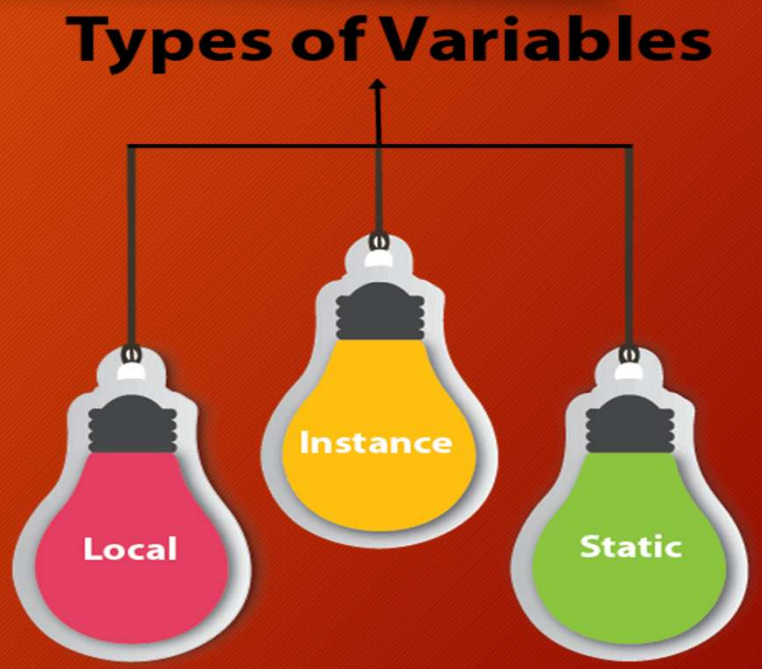
- A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.
- A variable is the name of a reserved area allocated in memory. In other words, it is a name of the memory location. It is a combination of "vary + able" which means its value can be changed.
- Variable is a name of memory location. There are three types of variables in java: local, instance and static.
- There are two types of data types in Java: primitive and non-primitive



Types of Variables

There are three types of variables in Java:

- local variable
- instance variable
- static variable



Types of Variables

1) Local Variable

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

2) Instance Variable

A variable declared inside the class but outside the body of the method, is called an instance variable. It is not declared as static.

It is called an instance variable because its value is instance-specific and is not shared among instances.

3) Static variable

A variable that is declared as static is called a static variable. It cannot be local. You can create a single copy of the static variable and share it among all the instances of the class. Memory allocation for static variables happens only once when the class is loaded in the memory.